Software Engineering & Project Management (SEPM)

Energy & Utility Management System (EUMS)

Electricity · Water · Gas — billing, metering and customer service

System Architecture

Software process model: Waterfall · Combined SEPM + DBMS course project

Group number	Group __
Course and section	Software Engineering & Project Management (SEPM), Section __
Supervisor / instructor	Prof. _____
Institute	_____ Institute of Technology
Date	October 2026

#	Name	Registration number
1	Sumit P Surendran	20243590
2	Sreethan Reddy	20243550
3	Sushrut Lad	20243551
4	Viswajith	20243559

1. System Architecture

1.1 Architecture diagram

EUMS uses a **three-tier (layered) client–server architecture** with the Model-View-Controller idea: templates are the *view*, Flask route functions are the *controller*, and the SQLite tables with `billing.py` form the *model*. The browser sends ordinary HTTP GET/POST requests (HTML forms); the server returns finished HTML pages. One small JSON endpoint gives the previous meter reading to the reading form.

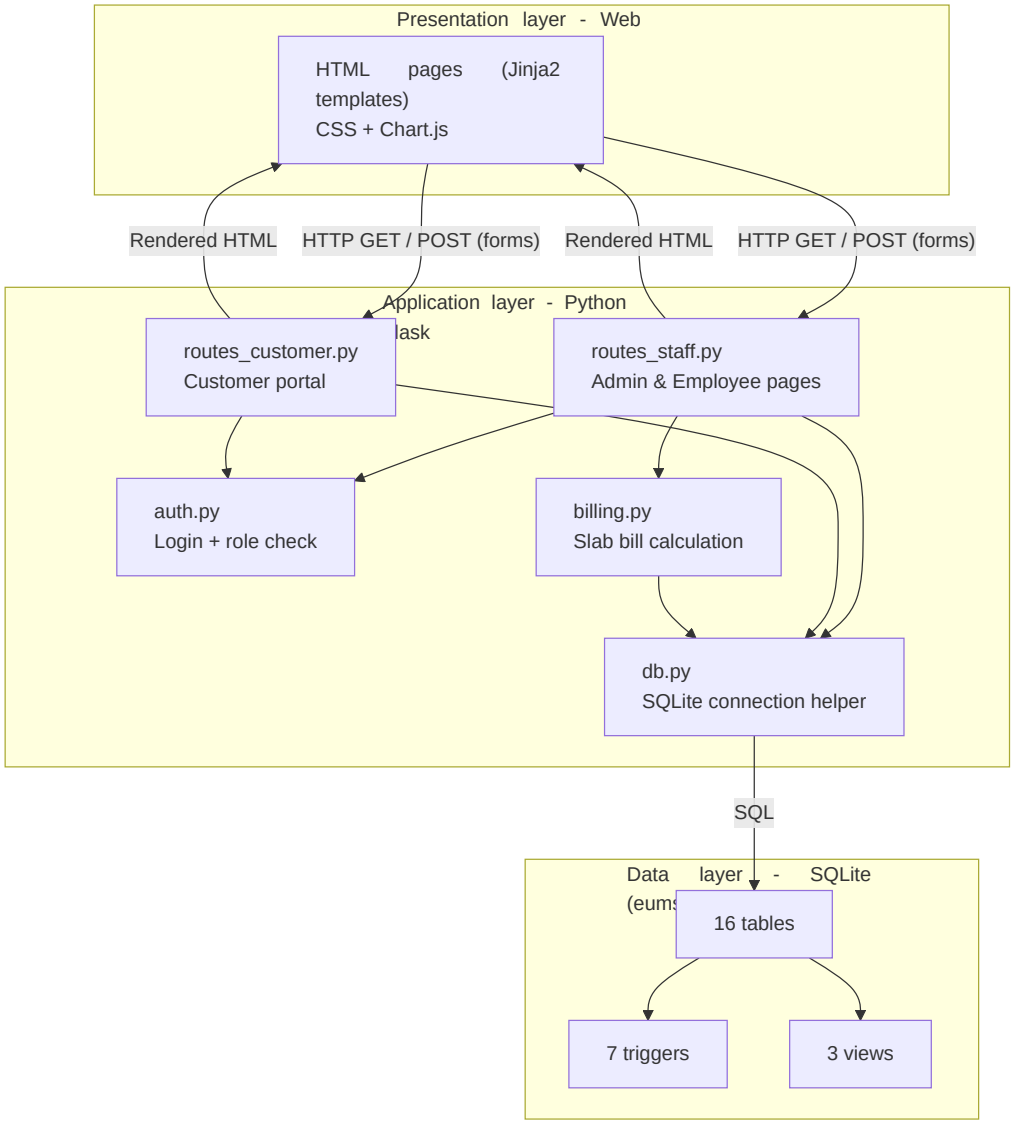


Figure 1: Three-tier architecture of EUMS

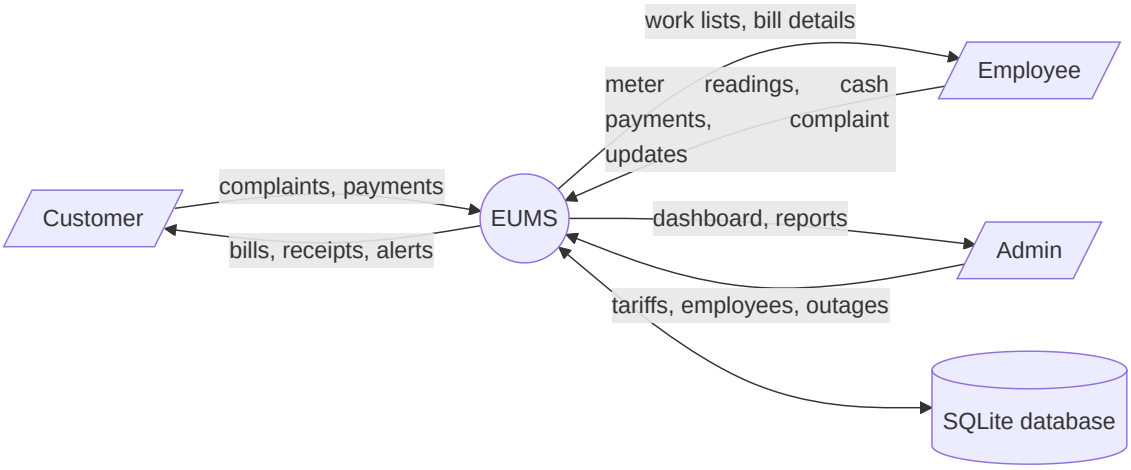


Figure 2: Context-level (Level 0) data flow diagram

1.2 Module description

File / module	Responsibility
`app.py`	Creates the Flask app, loads configuration, registers the two route groups, login/logout pages, error pages, and template helpers (money, date formatting).
`auth.py`	`login_required(*roles)` decorator: redirects to login if not signed in, returns 403 if the role is not allowed.
`db.py`	Opens one SQLite connection per request (foreign keys on, rows as dictionaries) and closes it afterwards; `query()` helper.
`billing.py`	Pure function `calculate_bill()` for slab pricing (unit tested) and `generate_bill()`, which reads the tariff and inserts the bill and bill items.
`routes_staff.py`	Admin and employee pages: dashboard, customers, connections, meter readings, billing, payments, complaints, outages, tariffs, employees, reports.
`routes_customer.py`	Customer portal: dashboard, connections, bills, pay, usage history, complaints, outages, profile, notifications. Every query is filtered by the logged-in customer.
`schema.sql`	All tables, constraints, indexes, triggers and views.
`seed.py`	Creates the database and loads demo data (24 customers from `sample_data/customers.csv`, 6 months of readings, bills, payments, complaints, outages).
`templates/`, `static/`	Jinja2 HTML pages (one per feature), one CSS file, and a small Chart.js helper.
`tests/`	pytest unit tests (billing) and functional tests (whole app with a fresh database per test).

Table 1: Modules